

Improving Memory Management in Three.js Applications Running Inside Mobile WebViews

Technical Case Study

Author	Asit Singh
Document type	Technical case study
Date	January 2023
Publication status	Technical case study draft - illustrative benchmark dataset

Abstract

Three-dimensional web applications can perform reliably in desktop browsers but experience serious performance and stability problems when executed inside mobile WebViews. These issues become more visible when an application loads large environments, high-resolution textures, repeated models, or long procedurally generated routes.

This case study examines a memory-related stability problem in a Three.js-based driving application running inside Android and iOS WebViews. The application performed normally for short routes, but longer routes increased JavaScript and graphics memory consumption. On some mobile devices, particularly iOS devices, this caused frame-rate degradation, long loading times, and unexpected WebView termination.

The investigation showed that the application created and retained more route content than was required for the player's current position. Hiding distant objects improved rendering performance but did not release their geometries, materials, textures, or JavaScript references. A bounded sliding-window route-streaming approach was therefore designed. Instead of keeping the complete route active, the system retains the current route chunk, a small number of upcoming chunks, and optionally one previous chunk.

Keywords: Three.js, React Three Fiber, WebView, mobile games, memory management, route streaming, GPU resources

1. Introduction

Web technologies are increasingly used to build interactive experiences inside native mobile applications. A common approach is to load a web application inside an Android or iOS WebView so that much of the same code can be shared between platforms.

Real-time 3D applications require more careful resource management than ordinary web pages. A Three.js scene can contain geometries, materials, textures, lights, animations, audio objects, physics bodies, and many JavaScript references. Desktop browsers generally have access to more memory and stronger graphics hardware, while mobile WebViews operate under stricter limits.

The application examined in this case study was a driving experience developed using JavaScript, React, React Three Fiber, and Three.js. Its route was assembled from reusable 3D sections such as city roads, highways, and transition areas. Short routes worked correctly, but as route distance increased the application became slower and less stable.

The initial assumption was that route distance itself caused the crash. Further investigation showed that distance was only an indirect factor. The primary issue was the amount of route data, instantiated 3D content, and graphical resources retained in memory.

2. Application Background

The driving application used reusable route models representing city sections, highway sections, intersections, transition areas, environmental decorations, and roadside objects.

A route-building function selected and positioned these sections according to the required distance. Each route entry stored a model identifier, position, rotation, scale, distance range, connection information, texture reference, and environment type.

This design allowed many route combinations to be created from a limited asset set. The drawback was that longer routes produced more metadata and, in the original implementation, more instantiated 3D content.

3. Problem Description

The original implementation prepared most or all of the route before gameplay started. Although only a small part of the route was visible, information and assets for a much larger route remained in memory.

JavaScript memory was consumed by route arrays, object references, transformation data, React state, model references, animation objects, event listeners, caches, and asynchronous loading promises.

Graphics memory was consumed by vertex buffers, index buffers, textures, render targets, shader programs, materials, and shadow maps. Hiding an object or removing it from the scene did not automatically release these resources.

```
routeChunk.visible = false;  
scene.remove(routeChunk);
```

These operations control visibility and scene membership, but they do not guarantee that associated GPU resources are disposed.

4. Initial Investigation

- Short routes worked more reliably than long routes.
- Desktop browsers handled long routes better than mobile WebViews.
- iOS devices showed the problem more frequently.
- Reducing texture quality improved stability.
- Hiding distant chunks improved frame rate but did not fully prevent crashes.
- Aggressive unloading caused visible interruptions.
- Some resources remained referenced after route sections were no longer required.
- Repeated route sessions gradually increased memory usage.

These observations suggested accumulated memory pressure rather than a single incorrect function.

5. Early Approaches

5.1 Rendering fewer chunks

Rendering only nearby chunks reduced per-frame work but did not fully solve memory growth because hidden objects and their resources could still remain allocated.

5.2 Removing passed chunks immediately

Immediate removal reduced scene size but introduced visible loading when the next model was not ready. It also created problems near chunk boundaries and during backward movement.

5.3 Caching every loaded asset

Caching reduced duplicate downloads and parsing, but an unlimited cache retained every asset used during the session. The cache therefore required explicit ownership and size limits.

6. Proposed Sliding-Window Architecture

The proposed solution uses a sliding window centred around the player's current route position. A typical configuration retains one previous chunk, the current chunk, and two upcoming chunks. This keeps approximately three to four chunks active at any time.

- The active window advances only when the player crosses a chunk boundary.
- Chunks entering the window are loaded or instantiated.
- Chunks still inside the window are preserved.
- Chunks leaving the window are detached and released according to ownership rules.
- The number of active chunks can be adjusted based on speed, loading time, device capability, and asset complexity.

```
activeStart = currentChunkIndex - chunksBehind;  
activeEnd   = currentChunkIndex + chunksAhead;
```

7. Separating Route Metadata from 3D Instances

7.1 Lightweight metadata

The complete route may be represented as compact metadata without immediately creating Three.js objects.

```
const routeEntry = {  
  id: "route-chunk-15",  
  type: "highway",  
  position: [0, 0, -450],  
  rotation: [0, 0, 0],  
}
```

Technical Case Study - illustrative benchmarks must be replaced with measured data

```

    scale: [1, 1, 1],
    startDistance: 450,
    endDistance: 480,
  };

```

7.2 Runtime instances

Three.js objects are created only when their route entries enter the active window. Each chunk follows a defined lifecycle:

```
planned -> requested -> loaded -> active -> inactive -> released
```

8. Active-Window Calculation

```

function getActiveChunkRange({
  currentIndex,
  totalChunks,
  chunksBehind = 1,
  chunksAhead = 2,
}) {
  if (totalChunks <= 0) return [];

  const start = Math.max(0, currentIndex - chunksBehind);
  const end = Math.min(totalChunks - 1, currentIndex + chunksAhead);

  const indexes = [];
  for (let index = start; index <= end; index += 1) {
    indexes.push(index);
  }

  return indexes;
}

```

The manager compares the newly required indexes with the currently loaded indexes. Chunks are classified as add, retain, or remove. This prevents unnecessary destruction and recreation.

9. Chunk Manager

```

class RouteChunkManager {
  constructor({ loadChunk, releaseChunk }) {
    this.loadChunk = loadChunk;
    this.releaseChunk = releaseChunk;
    this.loadedChunks = new Map();
  }

  async update(requiredIndexes, routePlan) {
    const requiredSet = new Set(requiredIndexes);

    for (const index of requiredIndexes) {
      if (this.loadedChunks.has(index)) continue;

      const routeEntry = routePlan[index];
      if (!routeEntry) continue;

      const chunk = await this.loadChunk(routeEntry);
    }
  }
}

```

```

    if (!requiredSet.has(index)) {
      this.releaseChunk(chunk);
      continue;
    }

    this.loadedChunks.set(index, chunk);
  }

  for (const [index, chunk] of this.loadedChunks) {
    if (requiredSet.has(index)) continue;

    this.releaseChunk(chunk);
    this.loadedChunks.delete(index);
  }
}

```

A production implementation must also handle cancelled requests, failed loads, duplicate requests, route resets, component unmounting, and shared resource ownership.

10. Predictive Loading

Loading only the current chunk is insufficient because the next chunk may become visible before its assets are ready. Upcoming chunks should be prepared based on player speed and observed loading time.

```

required preload distance = player speed x average loading time

```

A safety margin should be added for slower devices and unstable network conditions.

11. Asset Caching

A controlled promise cache prevents duplicate simultaneous requests while allowing failures to be retried.

```

const modelPromiseCache = new Map();

function loadModelOnce(url, loader) {
  if (!modelPromiseCache.has(url)) {
    const promise = loader.loadAsync(url).catch((error) => {
      modelPromiseCache.delete(url);
      throw error;
    });

    modelPromiseCache.set(url, promise);
  }

  return modelPromiseCache.get(url);
}

```

Cached source assets and runtime instances must be treated differently. Shared resources are managed centrally, while instance-owned resources are released with the chunk that created them.

12. Resource Cleanup

```
function disposeMaterial(material, disposeTextures = false) {
  if (!material) return;

  if (disposeTextures) {
    for (const value of Object.values(material)) {
      if (value?.isTexture) value.dispose();
    }
  }

  material.dispose();
}

function disposeObject3D(root, options = {}) {
  const {
    disposeGeometry = true,
    disposeMaterials = true,
    disposeTextures = false,
  } = options;

  root.traverse((object) => {
    if (!object.isMesh) return;

    if (disposeGeometry) object.geometry?.dispose();
    if (!disposeMaterials) return;

    const materials = Array.isArray(object.material)
      ? object.material
      : [object.material];

    for (const material of materials) {
      disposeMaterial(material, disposeTextures);
    }
  });

  root.removeFromParent();
}
```

This logic must not be applied blindly. Disposing a shared geometry, material, or texture while another chunk still uses it can cause broken rendering.

13. Reference Counting

```
class ResourceRegistry {
  constructor() {
    this.references = new Map();
  }

  retain(resource) {
    const count = this.references.get(resource) ?? 0;
    this.references.set(resource, count + 1);
  }

  release(resource) {
    const count = this.references.get(resource);
    if (!count) return;
  }
}
```

```

    if (count > 1) {
      this.references.set(resource, count - 1);
      return;
    }

    this.references.delete(resource);
    resource.dispose?();
  }
}

```

14. React Three Fiber Considerations

- Store frame-level player position in a ref.
- Update React state only when the active chunk index changes.
- Avoid expensive route calculations on every frame.
- Ignore or cancel asynchronous loads after unmounting.
- Use stable chunk identifiers as React keys.

```

const previousChunkIndexRef = useRef(-1);

useFrame(() => {
  const nextChunkIndex = calculateCurrentChunkIndex(
    playerPositionRef.current,
  );

  if (nextChunkIndex === previousChunkIndexRef.current) return;

  previousChunkIndexRef.current = nextChunkIndex;
  setCurrentChunkIndex(nextChunkIndex);
});

```

15. Avoiding Visible Reloading

- Use stable identifiers for route chunks.
- Compare previous and next active ranges before making changes.
- Preserve chunks that exist in both ranges.
- Load upcoming chunks before releasing the oldest chunk.
- Do not clear the shared model cache during normal movement.
- Keep at least one extra chunk ahead of the visible area.
- Confirm that an asynchronously loaded chunk is still required before attaching it.

16. Texture, Geometry, and Draw-Call Optimisation

- Reduce unnecessary texture dimensions.
- Use GPU-compressed textures where supported.
- Reuse textures, geometries, and materials.
- Use texture atlases where practical.
- Apply instanced meshes for repeated roadside objects.

- Reduce polygon counts and transparent materials.
- Limit unnecessary shadow-casting objects.
- Select asset quality according to device capability.

17. Testing Methodology

Testing should be performed on real Android and iOS devices. The baseline and optimized builds must use the same route, asset quality, player speed, camera behaviour, and test duration.

17.1 Recommended scenarios

- Short, medium, and long routes at normal speed.
- Long route at maximum speed.
- Repeated forward and backward movement.
- Application background and resume.
- Slow network conditions.
- Repeated route restart.
- Extended gameplay session.
- Rapid route switching and temporary asset-loading failure.

17.2 Benchmark table

Important: The table below is filled with an illustrative worked dataset for layout and discussion. The numbers are not verified measurements and must be replaced before they are cited as experimental results.

Metric	Original implementation	Optimized implementation
Device model and operating system	Illustrative iPhone 12 / iOS 16.2	Illustrative iPhone 12 / iOS 16.2
Route distance	1,000 km	1,000 km
Total planned chunks	120	120
Maximum active chunks	120	4
Initial loading time	18.6 s	5.2 s
Average frame rate	31 FPS	54 FPS
Minimum frame rate	12 FPS	38 FPS
Peak JavaScript heap	412 MB	148 MB
Active geometries	1,846	94
Active textures	286	38
Chunk transition time	Not applicable - route loaded	70-160 ms

Technical Case Study - illustrative benchmarks must be replaced with measured data

	initially	
Crashes across repeated tests	3 of 5 illustrative runs	0 of 5 illustrative runs

17.3 Recording requirements

- Record at least five runs for each implementation.
- Report averages, minimums, maximums, and test count.
- State the device, OS, WebView/browser version, build version, texture resolution, and test duration.
- Clear or preserve caches consistently across comparative runs.
- Define exactly what counts as a crash or failed run.
- Keep screenshots or profiler exports as supporting evidence.

18. Expected Behaviour

- Active 3D content remains bounded even when total route length increases.
- Startup time decreases because the full route is not instantiated.
- Upcoming sections are prepared before becoming visible.
- Passed sections are released according to ownership rules.
- Shared assets are not repeatedly downloaded.
- Memory usage remains more stable during extended sessions.
- Route transitions occur without obvious disappearance or reloading.

19. Limitations

This case study focuses on route streaming and graphical resource management. It does not fully evaluate physics memory, audio memory, native application memory, network architecture, battery usage, thermal throttling, or every WebView and GPU version.

The proposed active-window size is not universal and must be tuned using real device tests. The approach also assumes that route sections can be loaded independently.

20. Future Work

- Adaptive chunk-window size based on frame time and device capability.
- Automatic low-memory asset selection.
- Dynamic level of detail.
- GPU-compressed texture delivery.
- Worker-based or incremental route planning.
- Bounded least-recently-used asset caching.
- Automated memory regression tests.
- Predictive loading based on direction and speed.

Technical Case Study - illustrative benchmarks must be replaced with measured data

- Runtime quality reduction under memory pressure.

21. Lessons Learned

Visibility and memory ownership are different concerns. An invisible object can continue consuming JavaScript and graphics memory, and removing a React component or Three.js object does not guarantee that all associated resources have been released.

requested -> downloaded -> parsed -> cached -> cloned -> rendered -> detached -> disposed

Aggressive unloading is not always the best solution. A successful streaming architecture balances memory usage with uninterrupted gameplay. A cache is also not automatically an optimisation; without clear size and ownership rules, it can become another source of retained memory.

22. Conclusion

Long-route instability was not caused simply by numerical route distance. The main cause was the amount of route data and 3D content created or retained for that distance.

The proposed architecture replaces full-route instantiation with a bounded sliding-window model. Only a small number of nearby chunks remain active, lightweight metadata represents the remaining route, and upcoming sections are prepared before they become visible.

This approach provides a practical foundation for improving the stability of large Three.js experiences inside mobile WebViews. Final performance claims must be based on measurements collected from real devices.

23. Research Contribution and Scope

This manuscript does not claim that sliding-window streaming is a newly invented algorithm. Its contribution is an applied engineering design for bounded route streaming in a Three.js and React Three Fiber application embedded in mobile WebViews. The work combines route metadata separation, predictive loading, bounded active-scene ownership, controlled caching, and explicit resource-lifecycle management into one implementation-oriented case study.

The scope is limited to client-side route rendering and asset lifecycle. The document does not claim general performance results across all mobile devices, WebView versions, route assets, or network environments.

24. Reproducible Experimental Protocol

The following protocol must be completed before the manuscript contains performance claims. The baseline and optimized builds must differ only in the route-loading and resource-lifecycle implementation being evaluated.

Field	Required value	Publication status
Application/build identifier	Commit hash, release tag, or build number	Required before publication
Baseline definition	Exact behaviour of the original full-route implementation	Required before publication
Optimized definition	Exact window size, cache policy, and disposal policy	Required before publication
Android test device	Model, RAM, OS version, WebView version	Required for Android claims
iOS test device	Model, RAM, iOS version, WKWebView/Safari version	Required for iOS claims
Route configuration	Distance, chunk count, chunk dimensions, asset set	Required before publication
Asset configuration	Model sizes, polygon counts, texture dimensions and formats	Required before publication
Network condition	Offline/local, Wi-Fi, throttling profile, or measured throughput	Required before publication
Test duration	Duration per run and warm-up duration	Required before publication
Number of runs	Minimum five independent runs per build/device	Required before publication
Cache state	Cold-cache and/or warm-cache procedure	Required before publication
Measurement tools	Chrome DevTools, Safari Web Inspector, renderer.info, native logs, etc.	Required before publication
Crash definition	Observable condition used to classify a failed run	Required before publication

25. Results Reporting Template

The following table uses the same illustrative worked dataset as Section 17.2. It demonstrates reporting format only and is not evidence of measured performance.

Metric	Baseline	Optimized	How measured
Initial load time	18.6 s	5.2 s	Illustrative: navigation start to playable state
Average frame rate	31 FPS	54 FPS	Illustrative: fixed route segment over equal duration
1% low or minimum frame rate	12 FPS minimum	38 FPS minimum	Illustrative: frame-time trace
Peak JavaScript heap	412 MB	148 MB	Illustrative: profiler snapshot/trace
Three.js geometry count	1,846	94	Illustrative: <code>renderer.info.memory.geometries</code>
Three.js texture count	286	38	Illustrative: <code>renderer.info.memory.textures</code>
Maximum active chunks	120	4	Illustrative: chunk-manager instrumentation
Chunk transition latency	Not applicable	70-160 ms	Illustrative: timestamped transition events
Successful runs	2 of 5	5 of 5	Illustrative completed runs / total runs
Crash or forced termination	3 of 5	0 of 5	Illustrative native and WebView logs

26. Data Analysis Plan

- Use the same route, camera, player speed, asset quality, and test duration for both builds.
- Perform at least five runs per build on each claimed device.
- Report raw values as well as summary values; do not report only the best run.
- Report mean and range for load time, frame rate, heap usage, and transition latency.

- Separate cold-cache and warm-cache results.
- Record failed runs and crashes instead of excluding them from the average.
- Describe any outlier-removal rule before examining the results.
- Avoid claiming causation beyond what the controlled comparison supports.

27. Threats to Validity

27.1 Internal validity

Differences in cache state, network conditions, thermal state, background applications, route randomness, or asset versions can affect the comparison. These factors must be controlled or reported.

27.2 External validity

Results obtained on one or two devices cannot be generalized to every Android or iOS device. The findings apply primarily to the tested build, asset set, and device configurations.

27.3 Measurement validity

JavaScript heap measurements do not necessarily represent total GPU or native-process memory. The manuscript should therefore report tool-specific measurements and avoid describing them as total device memory.

28. Data and Code Availability

Code availability: [Choose one before publication: public repository URL / archived source snapshot / private company repository not publicly available].

Benchmark data availability: [Choose one before publication: attach CSV and profiler exports / publish an archive link / state that proprietary restrictions prevent release].

Asset availability: [State whether 3D models and textures are original, licensed, proprietary, or publicly available].

29. Ethics, Privacy, and Intellectual Property

This work does not involve human participants, personal data, medical data, or animal research. No ethics-board approval is expected to be required for the described software-performance evaluation.

Before publication, the author must confirm that the source code, screenshots, product names, employer details, 3D models, textures, and profiler outputs may legally be disclosed. Proprietary or client-owned material must be removed, anonymized, or published only with written permission.

30. Conflict of Interest and Funding

Conflict of interest: The author declares no known financial or personal conflict of interest related to this manuscript, unless a specific relationship is disclosed here before publication.

Funding: No external research funding is declared. If the work was completed as part of paid employment, state the employer and obtain permission where required.

31. Author Contributions

Conceptualization: Asit Singh. Software design and implementation: [confirm name(s)]. Testing and data collection: [confirm name(s)]. Data analysis: [confirm name(s)]. Writing - original draft: [confirm name(s)]. Writing - review and editing: [confirm name(s)].

Do not publish the contribution statement until each named person has approved the description.

32. Acknowledgements

The author may acknowledge colleagues or reviewers who provided technical feedback but did not qualify for authorship. Obtain permission before naming an individual or organization. If there are no acknowledgements, state: "The author has no additional acknowledgements."

33. Publication Metadata

Metadata field	Value
Title	Improving Memory Management in Three.js Applications Running Inside Mobile WebViews
Author	Asit Singh - confirm legal authorship before publication
Document type	Technical case study
Version	1.0 publication candidate
Original date	January 2023 - retain only if factually correct
Current revision date	June 2026
Peer-review status	Not peer reviewed
Suggested licence	All rights reserved, or select a Creative Commons licence after reviewing asset and employer rights
Suggested citation	Singh, A. (2023). Improving Memory Management in Three.js Applications Running

Technical Case Study - illustrative benchmarks must be replaced with measured data

34. Sources Consulted

- [1] Three.js. “How to dispose of objects.” Three.js Manual. No individual author or publication date stated; accessed 19 June 2026. <https://threejs.org/manual/#en/how-to-dispose-of-objects>. DOI: none.
- [2] Three.js. “Info.” Three.js Documentation. No individual author or publication date stated; accessed 19 June 2026. <https://threejs.org/docs/pages/Info.html>. DOI: none.
- [3] PMNDRS. “Performance pitfalls.” React Three Fiber Documentation. No individual author or publication date stated; accessed 19 June 2026. <https://r3f.docs.pmnd.rs/advanced/pitfalls>. DOI: none.
- [4] PMNDRS. “Scaling performance.” React Three Fiber Documentation. No individual author or publication date stated; accessed 19 June 2026. <https://r3f.docs.pmnd.rs/advanced/scaling-performance>. DOI: none.
- [5] PMNDRS. “Hooks.” React Three Fiber Documentation. No individual author or publication date stated; accessed 19 June 2026. <https://r3f.docs.pmnd.rs/api/hooks>. DOI: none.